Funkcje w PHP cz2. – Witryny i Aplikacje 2020

Zwracanie wartości przez funkcje

Przyjmowanie argumentów to nie jedyna cecha funkcji — mogą one również zwracać różne wartości. Czynność ta jest wykonywana za pomocą instrukcji return. Jeśli wystąpi ona wewnątrz funkcji, ta jest przerywana i zwraca wartość występującą po return.

```
function nazwa_funkcji (argumenty)
{
//instrukcje wnętrza funkcji
return wartość;
}
```

Jeśli wywołamy tego typu funkcję, to w miejscu jej wywołania zostanie wstawiona zwrócona przez nią wartość, która będzie mogła być wykorzystana w dalszej części skryptu.

Przykład: Funkcja dodająca argumenty i zwracająca wynik działania

```
<?php
function dodaj($wart1, $wart2)
{
$wynik = $wart1 + $wart2;
return $wynik;
}
$wartosc = dodaj (10, 20);
echo "Wynikiem dodawania 10 + 20 jest $wartosc.";
?>
```

Wynik skryptu: Wynikiem dodawania 10 + 20 jest 30.

Użycie samej instrukcji return bez żadnych argumentów również powoduje przerwanie działania funkcji.

Np.

```
<?php
function func()
{
echo "Pierwsza instrukcja echo";
```

```
return;
echo "Druga instrukcja echo";
}
func();
?>
```

Wynik: Pierwsza instrukcja echo

Wniosek: druga instrukcja echo została przerwana.

Zasięg zmiennych Zmienne globalne

UWAGA – W PHP większość zmiennych ma tzw. zasięg globalny, czyli można było z nich bezpośrednio korzystać w każdym miejscu skryptu z wyjątkiem wnętrz funkcji

```
<?php
$liczba = 1;
function func()
{
echo("Wartość zmiennej \$liczba w funkcji func to $liczba. <br />");
}
func();
echo("Wartość zmiennej \$liczba poza funkcją func to $liczba. <br />");
?>
```

Wynik:

E_NOTICE : type 8 -- Undefined variable: liczba -- at line 5

Wartość zmiennej $liczba w funkcji func to .

Wartość zmiennej $liczba poza funkcją func to 1.

Zmienna $liczba ma zasięg globalny — została zdefiniowana poza funkcją Func(). W związku z tym funkcja ta nie ma do niej bezpośredniego dostępu i próba odwołania nie może zakończyć się sukcesem. Stąd wyświetlenie ostrzeżenia i brak wartości zmiennej w napisie będącym wynikiem działania pierwszej instrukcji echo. Druga instrukcja echo, która znajduje się poza funkcją, ma pełny dostęp do zmiennej $liczba, dlatego też działa prawidłowo.

Jak odwołać się do zmiennej globalnej wewnątrz funkcji?

1. **Sposób:**

   odwołanie się do superglobalnej (z ang. *superglobal*) tablicy $GLOBAL

Zawiera ona odwołania do wszystkich zdefiniowanych w skrypcie zmiennych o zasięgu globalnym. Nazwy tych zmiennych są indeksami tej tablicy. Schematyczne odwołanie będzie miało więc postać:

$GLOBALS['*nazwa_zmiennej*']

```
<?php
$liczba = 1;
function func()
{
$liczba = 20; //tu planowana zmiana wartosci
echo "Wartość zmiennej \$liczba w funkcji func to ";
echo $GLOBALS['liczba'];  //pobiera zewnetrzną
echo ".<br />";
}
func();
echo "Wartość zmiennej \$liczba poza funkcją func to $liczba. <br />";
?>
```

Wynik:

Wartość zmiennej $liczba w funkcji func to 1.
Wartość zmiennej $liczba poza funkcją func to 1.

W funkcji func wartość odczytana z tablicy $GLOBALS została bezpośrednio przekazana instrukcji echo. W drugiej instrukcji echo dwukrotnie występuje ciąg $liczba.

W pierwszym przypadku ma jednak zostać wyświetlona nazwa zmiennej, a więc PHP NIE powinien interpretować zapisu, dlatego ciąg poprzedzony jest znakiem lewego ukośnika \. W ten sposób powstała sekwencja specjalna \$
, dzięki której na ekran wyprowadzany jest znak $ (a po nim ciąg liczba). W drugim przypadku ciąg $liczba występuje bez żadnych dodatkowych znaków, a zatem (ponieważ całość jest ujęta w cudzysłów) PHP dokona interpretacji i w miejscu nazwy zmiennej wstawi jej wartość.

2 SPOSÓB - użycie słowa kluczowego global

Drugi sposób odwołania się do zmiennej globalnej wewnątrz funkcji wymaga użycia słowa kluczowego global. Przed odwołaniem należy użyć konstrukcji w postaci:

global $*nazwa_zmiennej*;

Można też w jednym wierszu zarejestrować więcej zmiennych, wtedy ich nazwy należy oddzielić przecinkami:

global $*nazwa_zmiennej_1*, $*nazwa_zmiennej_2*, $*nazwa_zmiennej_3*;

Użycie słowa kluczowego global

```
<?php
$liczba = 1;
function func()
{
global $liczba;
echo "Wartość zmiennej \$liczba w funkcji func to $liczba. <br />";
}
func();
echo "Wartość zmiennej \$liczba poza funkcją func to $liczba. <br />";
?>
```

INNE Przykłady praktyczne – kilka kodów z zajęć:

```
?php
$liczba = 1;
function func()
{
$liczba = 20; //tu planowana zmiana wartosci
echo "Wartość zmiennej liczba20 \$liczba w funkcji func to ";
echo $GLOBALS['liczba']; //pobiera zewnetrzną
echo ".<br />";
echo "Wartość zmiennej liczba22 \$liczba w funkcji func to "; echo ".<br />";

}
func();
echo "Wartość zmiennej \$liczba poza funkcją func to $liczba. <br />";
?>
```

```
<?php
$liczba = 1;
function func()
{
$liczba = 20; //tu planowana zmiana wartosci
echo "Wartość zmiennej liczba20 \$liczba w funkcji func to ";
echo $GLOBALS['liczba']; //pobiera zewnetrzną
echo ".<br />";
echo "Wartość zmiennej liczba22 \$liczba w funkcji func to ";
echo $GLOBALS['liczba']; //pobiera zewnetrzną
}
func();
echo "Wartość zmiennej \$liczba poza funkcją func to $liczba. <br />";
?>
```

nowszy kod do notatek

```
<?php
$liczba = 1;
function func()
{

echo "Wartość zmiennej PRZED \$liczba w funkcji func to $liczba. <br />";
global $liczba;
echo "Wartość zmiennej \$liczba w funkcji func to $liczba. <br />";
}
```

```
func();
echo "Wartość zmiennej \$liczba poza funkcją func to $liczba. <br />";
?>
<?php
$liczba = 1;
function func()
{

echo "Wartość zmiennej PRZED \$liczba w funkcji func to $liczba. <br />";
global $liczba;
echo "Wartość zmiennej wewnatrz \$liczba w funkcji func to $liczba. <br />";
$liczba = 10;
global $liczba;
$liczba = 11;
echo "Wartość zmiennej wewnatrz2 \$liczba w funkcji func to $liczba. <br />";


}
func();
echo "Wartość zmiennej \$liczba poza funkcją func to $liczba. <br />";
?>

<?php
function func()
{
echo "\$liczba = $liczba \n";
$liczba = 10;
echo "\$liczba = $liczba \n";
$liczba += 20; //nie jest zapamiętana
}
func();
func();
?>
```

.....

Zmienne lokalne

Oprócz zmiennych globalnych istnieją także zmienne lokalne funkcji. Jak sama nazwa wskazuje, ich zasięg jest ograniczony jedynie do wnętrza funkcji, w której zostały zdefiniowane. Próba odwołania w innym miejscu nie będzie możliwa.

```
<?php
function func()
{
$liczba = 10;
echo "Wartość zmiennej \$liczba w funkcji func to $liczba";echo ("<br>");
}
func();
echo "Wartość zmiennej \$liczba poza funkcją func to $liczba"; echo("<br>");
?>
```

Wynik:

```
Wartość zmiennej $liczba w funkcji func to 10. <br /><br />
<b>Notice</b>:  Undefined variable: liczba in <b>[...][...]</b> on line
<b>8</b><br />
Wartość zmiennej $liczba poza funkcją func to . <br />
```

Inny przykład:

```
<?php
$liczba = 1;
function func()
{
$liczba = 10;
echo "Wartość zmiennej \$liczba w funkcji func to $liczba \n";
}
func();
echo "\n Wartość zmiennej \$liczba poza funkcją func to $liczba \n";
?>
```

Wynik:
Wartość zmiennej $liczba w funkcji func to 10
 Wartość zmiennej $liczba poza funkcją func to 1

Dlaczego taki jest wynik?

Zmienne statyczne

Nazwa zmienna statyczna to takie zmienne lokalne funkcji, które zachowują swoją wartość pomiędzy wywołaniami funkcji. Zwykła zmienna lokalna jest widoczna tylko w obrębie funkcji i jest tworzona tylko w momencie wywołania tej funkcji. Gdy funkcja zakończy działanie, zmienna znika.
Trzeba zmiennej statycznej przypisać wartość początkową.

```
<?php
function func()
{
$liczba = 10;
echo "\$liczba = $liczba \n";
$liczba += 20;  //nie jest zapamiętana
}
func();
func();
?>
```

Wynik:
$liczba = 10
$liczba = 10

Sposoby przekazywania argumentów

Argumenty funkcji mogą być przekazywane na dwa sposoby:

- przez wartość (z ang. *by value*),

```
<?php
function dodajJeden($liczba)
{
$liczba = $liczba + 1;

echo "Wewnątrz funkcji \$liczba = $liczba \n";
}
$liczba = 3;
echo "Przed wywołaniem funkcji \$liczba = $liczba \n";
dodajJeden($liczba);
echo "Po wywołaniu funkcji \$liczba = $liczba \n";
```

```
?>
```

Co pokazuje ten przykład?

- za pomocą referencji (z ang. *by reference*).

Najczęściej modyfikujemy w funkcji argumentu oryginalnego. Wtedy trzeba zastosować sposób drugi — przekazywanie przez referencję. W tym celu przed nazwą argumentu należy umieścić znak & (ampersand).

```
<?php
function dodajJeden(&$liczba)  //tu jest zmiana
{
$liczba = $liczba + 1;
}
$liczba = 1;
echo("Przed wywołaniem funkcji \$liczba = $liczba \n");
dodajJeden($liczba);
echo("Po wywołaniu funkcji \$liczba = $liczba \n");
?>
```

W tym przypadku argument wejściowy od razu staje się wyjściowym.

```
<?php
function dodajJeden(&$liczba, &$liczba2)  //tu jest zmiana
{
$liczba = $liczba + 1;
$liczba2 = $liczba + 100;
}
$liczba = 1;
echo("Przed wywołaniem funkcji \$liczba = $liczba \n");
dodajJeden($liczba,$liczba2);
echo("Po wywołaniu funkcji \$liczba = $liczba \n");
echo("Po wywołaniu funkcji \$liczba2 = $liczba2 \n");
?>
```

Co pokazuje ten przykład?

Ćwiczenia praktyczne:

Zad.1

Przy pomocy przekazywania przez wartość i przekazywania przez referencję – na bazie poprzednich zadań wykonać funkcję:

- Na podstawie boku A – podanego w wywołaniu funkcji, funkcja oblicza i zwraca wyniki: - obwodu kwadratu, pola kwadratu, pola sześcianu, objętości sześcianu.

Zad2.

Przy pomocy przekazywania przez wartość i przekazywania przez referencję – na bazie poprzednich zadań wykonać funkcję:

Funkcja pobiera w wywołaniu wartości dwumianu kwadratowego – A, B, C i zwraca pierwiastki równania kwadratowego (oba) oraz zwraca wartość delty.
Gdy delta < 0 – pierwiastki zwracane równe zero.
Gdy delta = 0 – zwrot obu pierwiastków.